Multiple clock cycle datapath

(reference: text Sections 4.5, C.4)

in a single clock cycle datapath, every instruction gets the same time to complete (one clock cycle), even though less work is required for some instructions than for others

- e.g., "lw" versus "beq"

will take a **brief** look at the design of a multiple clock cycle datapath in which:

- each instruction is broken up into multiple steps, with instructions requiring more work using more steps
- instructions are executed one-by-one (as in the single clock cycle datapath), i.e. no pipelining

how should instructions be broken up into multiple steps?

- ideally, the work done in each step should take similar amounts of time to complete – since each step will be given the same time, one clock cycle!

one possible breakdown into steps for our subset of RISC-V instructions (a simple breakdown, but likely not ideal with respect to balancing work in each step!)

all would have the same first two steps:

1. fetch the instruction and in parallel
   increment PC
2. fetch register operands and in parallel
   decode the instruction

subsequent steps would depend on the instruction type, e.g. for "lw", could use three more steps:

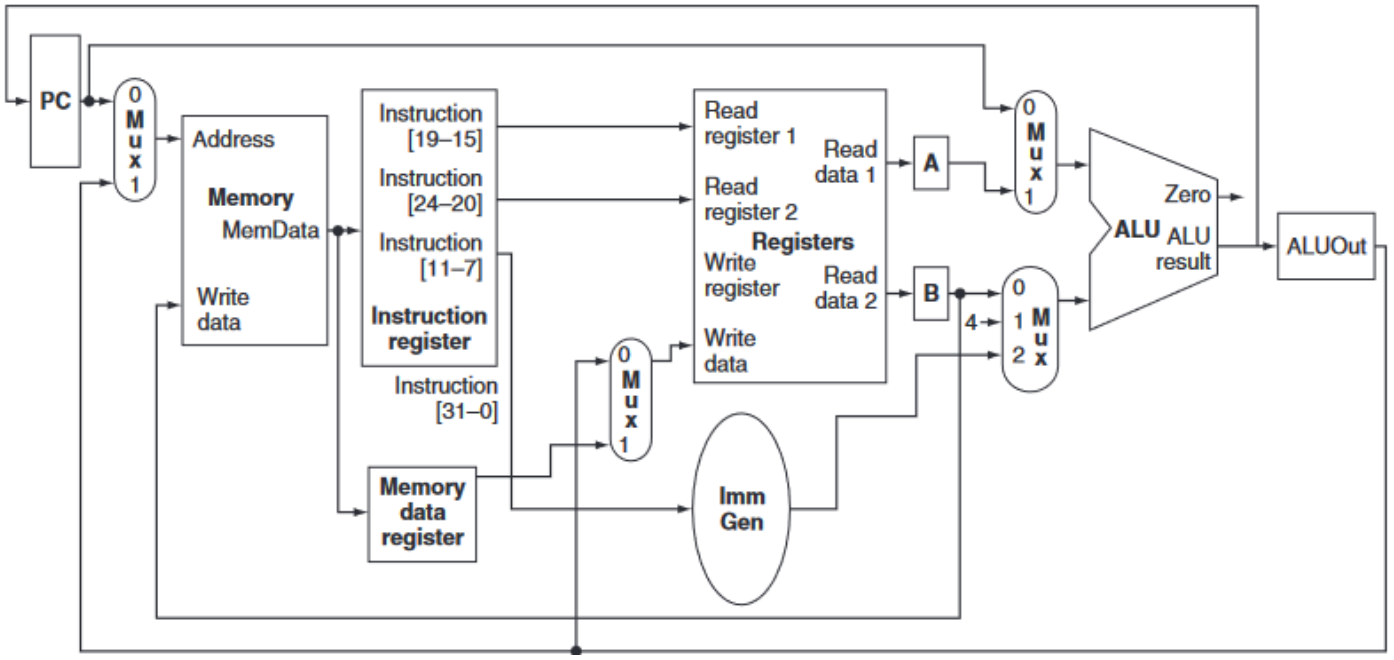
3. calculate memory address
4. read from the data memory
5. write into the register file

while for "beq", e.g., could use just one more step in which the branch target address is calculated and in parallel the register operands are compared

additional storage units will be needed to save the work we've done in one step for use in the next … for example, we will need an Instruction Register to hold the instruction fetched in step 1

multiple clock cycle datapath design for small subset of RISC-V instructions (add, sub, and, or, lw/sw, not beq)

(from text Fig. e4.5.2, but *bug in this figure*, see also Fig. e4.5.3)



no longer need separate adder for incrementing PC, or separate instruction and data memories … *why*?

new storage units:

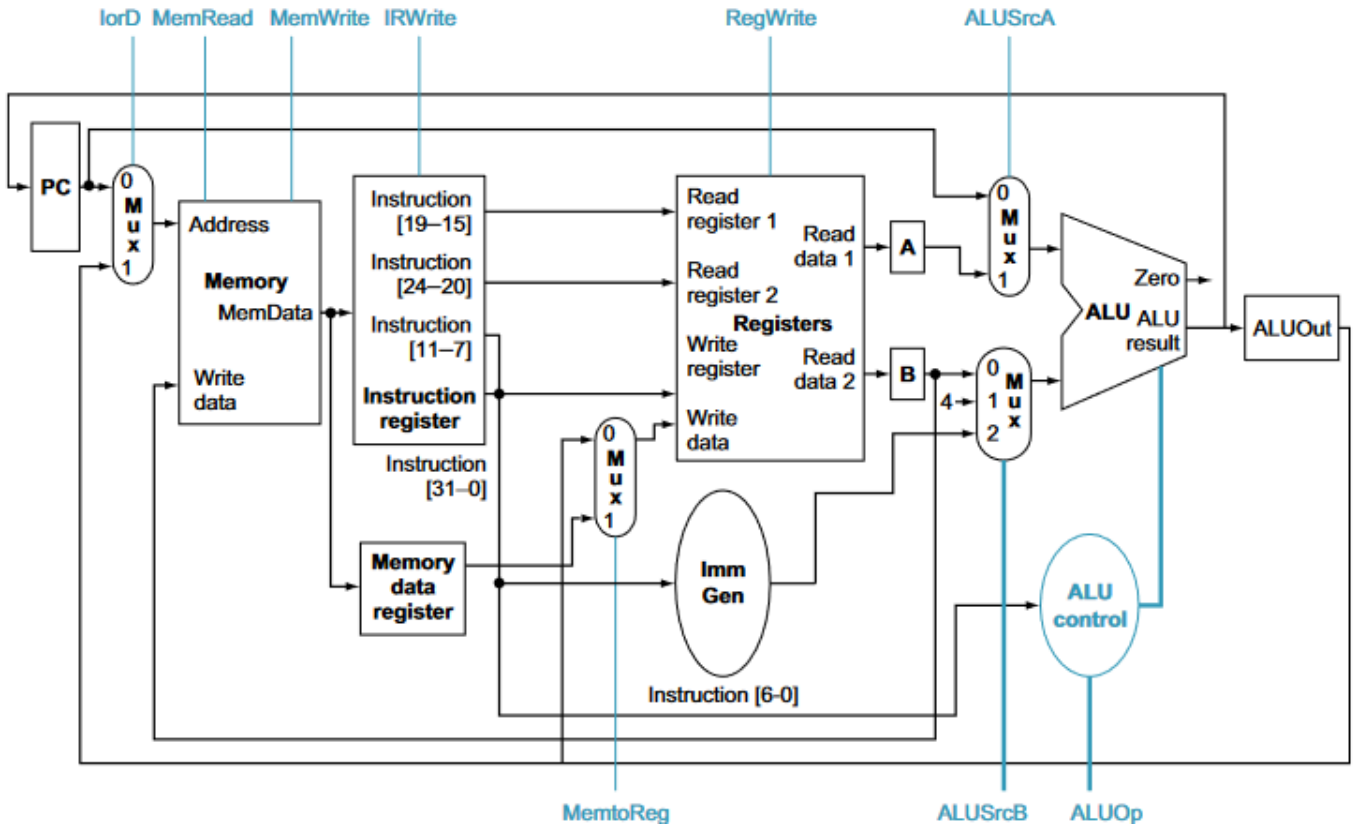
- Instruction Register
- Memory Data Register
- A & B registers
- ALUOut register

control for a multiple clock cycle datapath

more complex that for single clock cycle datapath

- what are all the different steps we will need?
- what control line settings are required in each?
- what is the sequence of steps followed for each instruction type?

(from text Fig. e4.5.3)



two basic approaches to specifying the required control: finite state machines and microprograms

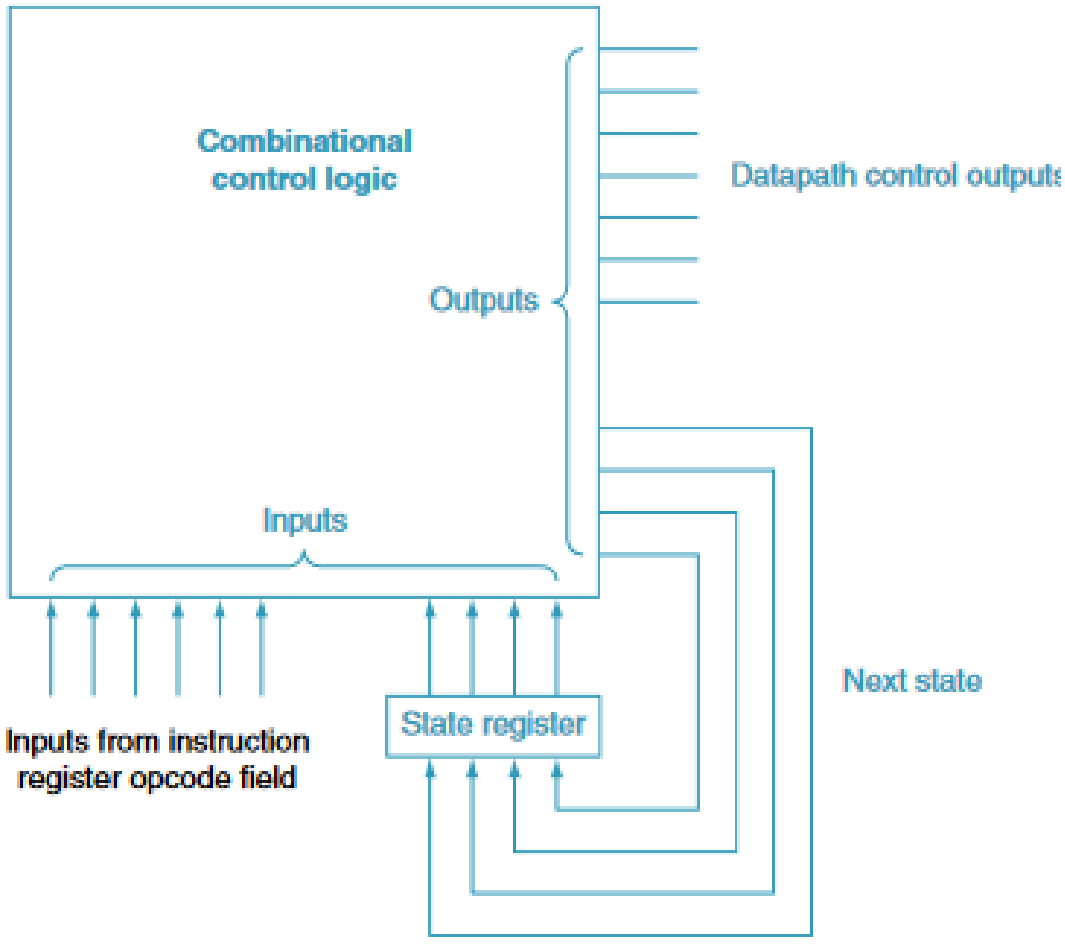
finite state machines

each of the different steps needed termed a “state”

instructions move from state to state as they execute

for “main control”, could use combinational logic circuit that takes as input a state number and the opcode field, and outputs the control line settings and the number of the next state

(from text Fig. e4.5.13)

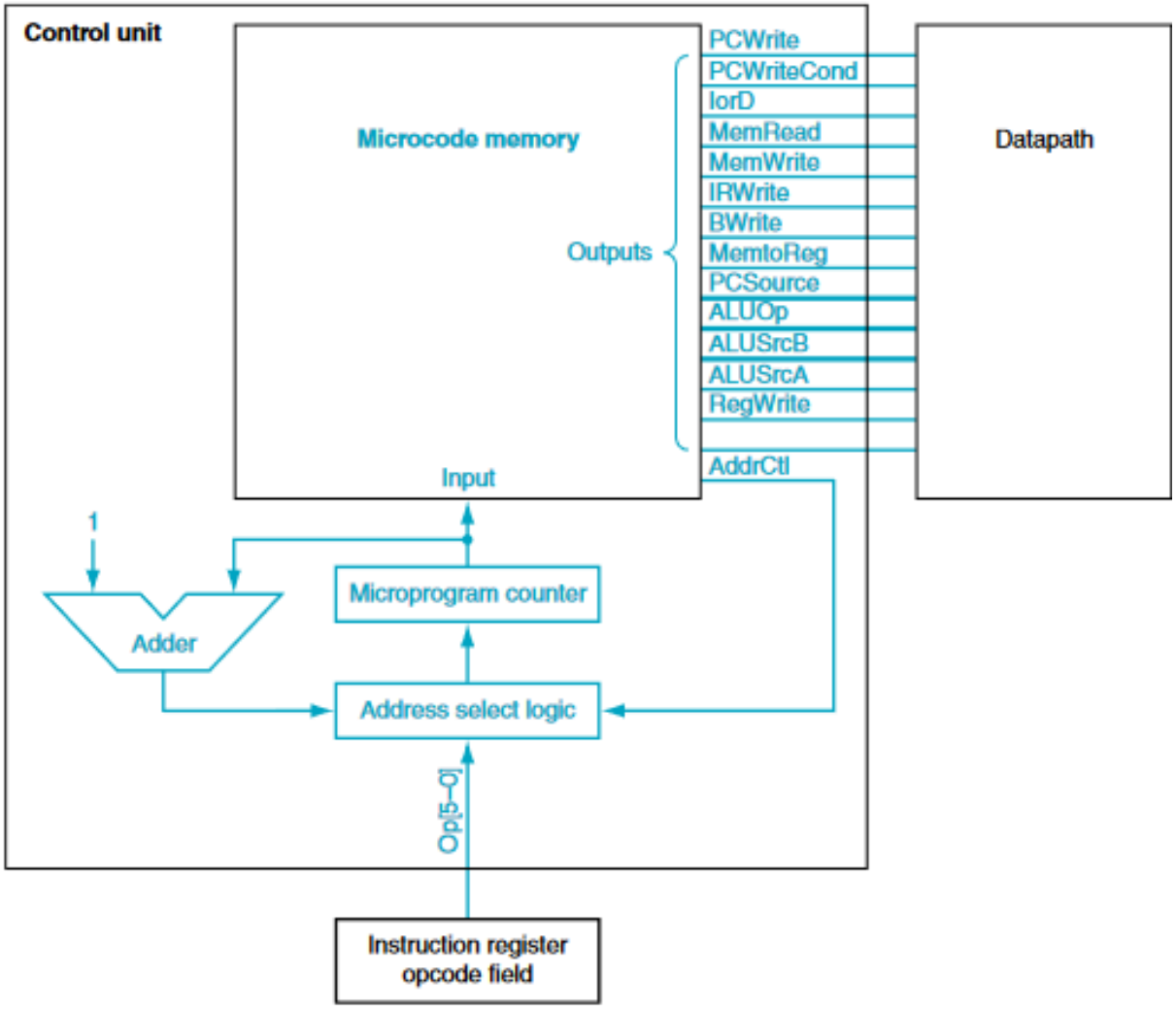


microprograms

instead of having a big combinational logic circuit that outputs the control line settings, **store** these settings in a special memory on the processor chip

- the stored settings for a particular state termed a "microinstruction"
- execute a machine language instruction by executing the corresponding "microprogram"

(from text Fig. C.4.6) (in this example have more control signals)



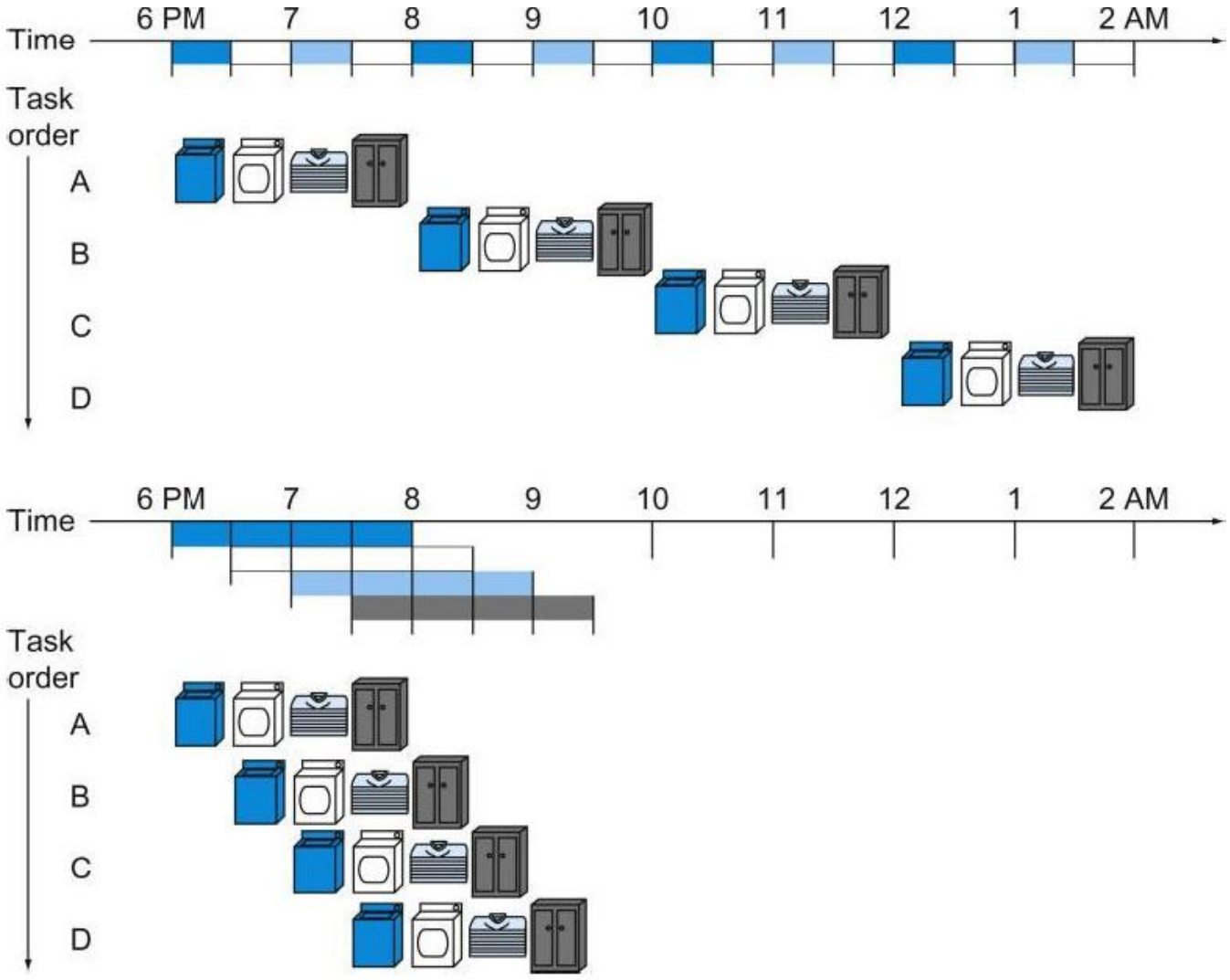
Pipelining

(reference: text Sections 4.6-4.11)

idea is to overlap the processing of different instructions, like in an assembly line

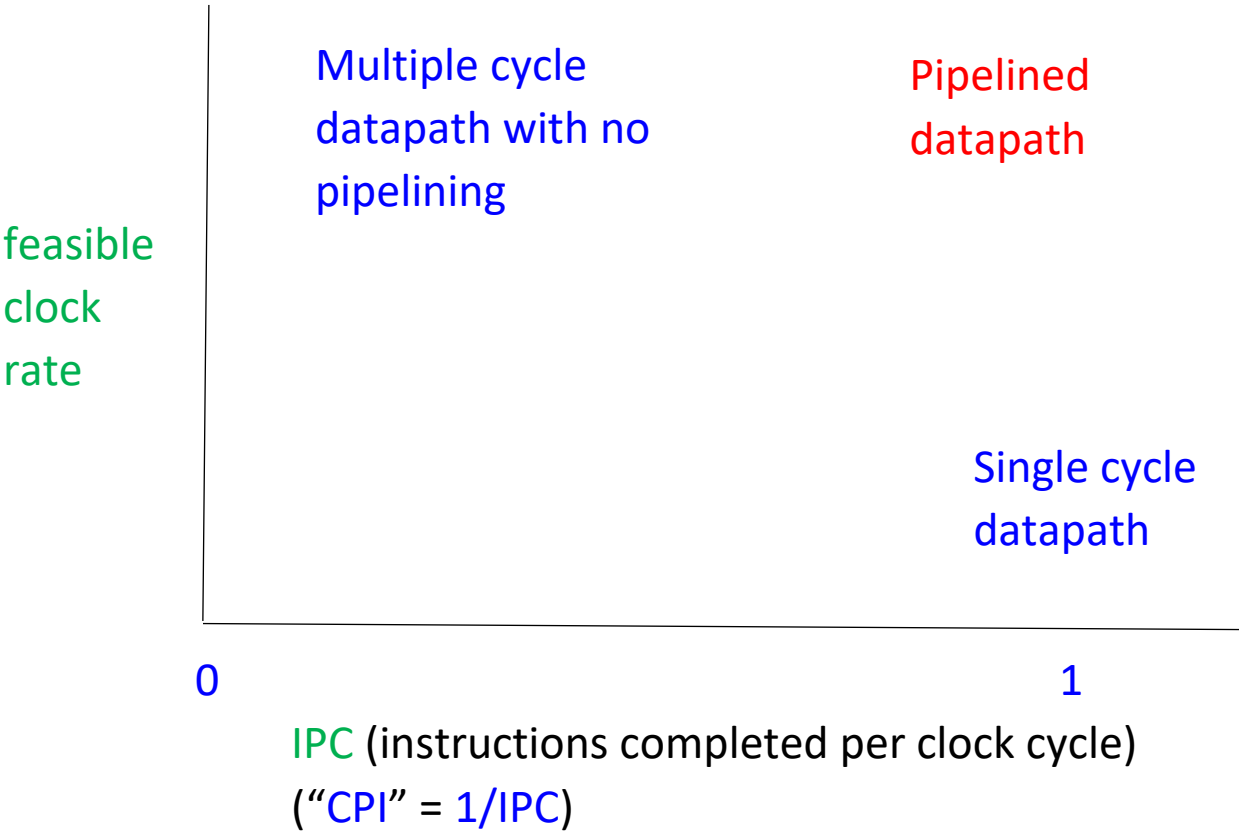
laundry analogy

(from text Fig. 4.27)



potential performance improvement with pipelining

consider IPC ("instructions completed per clock cycle")



single clock cycle datapath has IPC = 1, but needs very long clock cycle time

multiple clock cycle datapath can have much shorter clock cycle time (therefore much higher clock rate), but has much lower IPC

pipelined datapath can potentially get close to IPC = 1 (but not quite, owing to "hazards" that disrupt pipeline operation), with short clock cycle time (high clock rate)